

3

Dasar-dasar Pemrograman dalam Bahasa C

3.1 Mengubah algoritma menjadi program C

Pada dua kuliah sebelumnya kita telah membangun dua fondasi penting. Pertama, kita melihat hubungan

masalah → model → algoritma → program → hasil

Kedua, kita mempelajari bagaimana suatu persoalan dapat dirumuskan menjadi langkah-langkah algoritmik melalui dekomposisi, diagram alir, *pseudocode*, *tracing*, serta pemeriksaan kebenaran secara sederhana.

Pada kuliah ini kita mulai memusatkan perhatian pada bagian

algoritma → program C.

Saat ini sudah era AI sehingga kita sebenarnya tidak perlu menghafal sintaks bahasa C secara lengkap. Namun, kita perlu memahami bagaimana konsep algoritmik seperti

- ▶ masukan,
- ▶ penyimpanan nilai,
- ▶ perhitungan,
- ▶ keputusan logika, dan
- ▶ keluaran

direpresentasikan dalam bahasa C.

Bahasa C dipilih sebagai bahasa utama pada bagian awal mata kuliah karena beberapa alasan:

- ▶ C memiliki sintaks yang relatif kecil dan terstruktur.
- ▶ Hubungan antara program, tipe data, memori, dan representasi data cukup terlihat.

- ▶ C banyak digunakan dalam sistem tertanam, mikrokontroler, instrumentasi, sistem operasi, komputasi berperforma tinggi, dan perangkat lunak rekayasa.
- ▶ Banyak bahasa pemrograman modern menggunakan konsep sintaks yang dipengaruhi oleh C.
- ▶ C memaksa kita memahami beberapa konsep komputasi secara eksplisit, misalnya tipe data, alamat memori, dan proses kompilasi.

Bagi mahasiswa Teknik Fisika, bahasa C juga relevan karena banyak sistem akuisisi data, perangkat instrumentasi, mikrokontroler, dan sistem kendali menggunakan C atau bahasa yang sangat dekat dengan C.

3.1.1 Algoritma yang sama, bahasa yang berbeda

Misalkan kita ingin menghitung energi kinetik,

$$E_k = \frac{1}{2}mv^2.$$

Secara algoritmik,

```
INPUT  $m$ 
INPUT  $v$ 
 $E_k \leftarrow 0.5 \cdot m \cdot v \cdot v$ 
OUTPUT  $E_k$ 
```

Dalam C, kita dapat menulis

```
double kinetic_energy = 0.5 * mass * velocity * velocity;
```

Dalam Python, algoritma yang sama dapat ditulis

```
kinetic_energy = 0.5 * mass * velocity**2
```

Sintaksnya berbeda, tetapi algoritmanya sama.

Fakta ini penting dipahami karena pemrograman tidak identik dengan bahasa pemrograman tertentu. Bahasa pemrograman adalah media untuk mengekspresikan algoritma.

3.2 Struktur dasar program C

Perhatikan program C berikut.

```
#include <stdio.h>

int main(void)
{
    printf("Hello, Teknik Fisika!\n");

    return 0;
}
```

Program tersebut terdiri atas beberapa bagian penting.

3.2.1 Direktif `#include`

Baris `#include <stdio.h>` meminta *preprocessor* C untuk menyertakan deklarasi dari pustaka standar input-output.

`stdio` merupakan singkatan dari *standard input/output* sedangkan `.h` menunjukkan *header file*. Kita memerlukan `stdio.h` ketika menggunakan fungsi seperti `printf()` dan `scanf()`. Secara konseptual, pustaka standar menyediakan fungsi-fungsi yang telah ditulis dan diuji sehingga kita tidak perlu membuat semuanya dari awal.

3.2.2 Fungsi `main`

Setiap program C yang dapat dieksekusi memiliki titik awal eksekusi. Dalam contoh kita, `int main(void)` menyatakan fungsi utama program.

Kata `int` menunjukkan bahwa fungsi `main` mengembalikan (*return*) atau memberikan sebuah nilai bertipe integer kepada lingkungan yang menjalankan program. Bagian `(void)` menyatakan bahwa fungsi tersebut tidak menerima argumen. Tubuh fungsi dituliskan di antara kurung kurawal,

```
{  
    ...  
}
```

Program mulai menjalankan pernyataan-pernyataan di dalam `main` dari atas menuju bawah, kecuali aliran kontrol mengubah urutannya.

3.2.3 Pernyataan (*statement*)

Baris

```
printf("Hello, Teknik Fisika!\n");
```

merupakan suatu **pernyataan** (*statement*). Dalam C, banyak pernyataan diakhiri dengan titik koma, `;`. Titik koma merupakan bagian dari sintaks. Jika kita menulis

```
printf("Hello, Teknik Fisika!\n")
```

tanpa titik koma, *compiler* akan menghasilkan galat sintaksis.

3.2.4 Akhir program atau subprogram, `return 0`

Baris

```
return 0;
```

mengakhiri bagian utama program, yakni “fungsi” `main`, dan mengembalikan nilai `0`. Secara konvensi, nilai nol menunjukkan bahwa program selesai secara normal.

Program atau nantinya subprogram secara umum dapat menggunakan nilai selain nol untuk menunjukkan kondisi gagal atau kesalahan. Namun, kita akan menggunakan bentuk

```
return 0;
```

secara konsisten untuk program-program sederhana.

3.3 Kompilasi program

Program C tidak langsung dijalankan dari kode sumber. Kita telah melihat alur

```
source code → compiler → executable → execution.
```

Misalkan program disimpan sebagai `hello.c`. Kita dapat mengompilasinya menggunakan GCC. Pada Linux atau WSL,

```
gcc -Wall -Wextra hello.c -o hello
```

Kemudian, program dijalankan dengan

```
./hello
```

Sementara itu, pada Windows dengan GCC, kita dapat menuliskan

```
gcc -Wall -Wextra hello.c -o hello.exe
```

dan dapat dijalankan dengan

```
.\hello.exe
```

Opsi

```
-Wall  
-Wextra
```

meminta *compiler* menampilkan lebih banyak peringatan. Sepanjang mata kuliah ini, peringatan *compiler* harus diperlakukan sebagai informasi penting, bukan sebagai gangguan. Program yang dapat dikompilasi belum tentu benar, tetapi program yang menghasilkan banyak peringatan sebaiknya diperiksa sebelum dilanjutkan.

3.4 Komentar dalam C

Komentar digunakan untuk memberikan informasi kepada manusia yang membaca program. Komentar tidak akan dieksekusi oleh komputer. Komentar satu baris ditulis menggunakan `// komentar satu baris`. Komentar beberapa baris dapat ditulis menggunakan

```
/*  
    komentar  
    beberapa baris  
*/
```

Sebagai contoh,

```
#include <stdio.h>  
  
int main(void)
```

```
{  
    // Massa dalam kilogram  
    double mass = 2.0;  
  
    // Kecepatan dalam meter per sekon  
    double velocity = 3.0;  
  
    double energy = 0.5 * mass * velocity * velocity;  
  
    printf("Kinetic energy = %f J\n", energy);  
  
    return 0;  
}
```

Komentar sebaiknya menjelaskan sesuatu yang tidak langsung terlihat dari kode. Komentar seperti

```
x = x + 1; // tambah x dengan 1
```

sering kali tidak memberikan informasi tambahan. Sementara itu, komentar semacam

```
double velocity = 20.0; // m/s
```

berguna karena menjelaskan satuan.

3.5 Variabel dan nilai

Dalam program, kita sering perlu menyimpan nilai yang dapat digunakan kembali. Tempat penyimpanan bernama disebut **variabel**. Sebagai contoh,

```
double mass = 2.0;
```

mendeklarasikan variabel bernama `mass` dengan tipe `double` dan memberikan nilai awal 2.0. Secara konseptual,



Nama `mass` digunakan agar kita tidak perlu memikirkan alamat memori secara langsung.

3.5.1 Deklarasi

Pernyataan

```
double mass;
```

disebut deklarasi variabel. Deklarasi memberitahu *compiler* bahwa kita ingin menggunakan variabel bernama `mass` dengan tipe `double`. Jika kita ingin langsung memberikan nilai awal,

```
double mass = 2.0;
```

proses pemberian nilai awal ini disebut **inisialisasi**.

3.5.2 Assignment (penetapan)

Setelah variabel dibuat, nilainya dapat diperbarui.

```
mass = 3.5;
```

Operasi tersebut disebut **assignment** atau pemberian nilai.

Perhatikan bahwa simbol `=` dalam C bukan merupakan tanda kesamaan matematika. Dalam C,

```
x = 5;
```

berarti

simpan nilai 5 ke dalam variabel `x`.

Oleh karena itu, pernyataan

```
x = x + 1;
```

valid dalam C dan dalam mayoritas bahasa pemrograman. Maksudnya adalah

1. ambil nilai lama `x`;
2. tambahkan satu;
3. simpan hasilnya kembali ke `x`.

Misalkan variabel `x` nilai awalnya

$x = 4.$

Setelah

```
x = x + 1;
```

nilai barunya adalah

$x = 5.$

3.5.3 Variabel harus diberi nilai sebelum digunakan

Perhatikan program berikut.

```
#include <stdio.h>

int main(void)
{
    double temperature;

    printf("%f\n", temperature);

    return 0;
}
```

Variabel `temperature` dibuat tetapi belum diberi nilai.

Penggunaan nilai variabel lokal yang belum diinisialisasi menghasilkan perilaku yang tidak dapat diandalkan. Oleh karena itu, biasanya memberikan nilai sebelum variabel digunakan.

```
double temperature = 25.0;
```

Kebiasaan inisialisasi yang jelas akan mengurangi banyak kesalahan komputasi.

3.6 Aturan penamaan variabel

Nama variabel dalam C disebut **identifier**. Contoh *identifier* yang valid adalah

```
mass
velocity
temperature
sensor_value
x0
pressure_1
```

Secara umum, *identifier* dapat menggunakan

- ▶ huruf;
- ▶ angka;
- ▶ garis bawah `_`.

Namun, *identifier* tidak boleh dimulai dengan angka. Contoh yang tidak valid:

```
2mass
3temperature
```

C bersifat **case-sensitive**, alias sensitif dengan besar-kecilnya huruf. Artinya,

```
temperature
Temperature
TEMPERATURE
```

dianggap sebagai tiga nama yang berbeda.

3.6.1 Gunakan nama yang bermakna

Bandingkan

```
double a = 12.0;  
double b = 2.5;  
double c = a * b;
```

dengan

```
double voltage = 12.0;  
double current = 2.5;  
double power = voltage * current;
```

Keduanya dapat menghasilkan nilai yang sama, tetapi program kedua jauh lebih mudah dipahami.

Untuk program teknik, nama variabel sebaiknya memberikan informasi mengenai makna fisis. Beberapa contoh:

```
double mass;  
double velocity;  
double temperature;  
double resistance;  
double pressure;  
double time_step;
```

Jika diperlukan, satuan dapat dicantumkan dalam komentar atau bahkan nama variabel.

```
double temperature_celsius = 25.0;  
double pressure_pa = 101325.0;
```

Pendekatan tersebut sangat membantu ketika beberapa sistem satuan digunakan sekaligus.

3.7 Tipe data dasar

Tipe data akan menentukan

- ▶ jenis nilai yang dapat disimpan;
- ▶ cara nilai direpresentasikan;
- ▶ operasi yang dapat dilakukan;
- ▶ jumlah memori yang biasanya digunakan.

Beberapa tipe data dasar yang penting adalah

- ▶ `char`,
- ▶ `int`,
- ▶ `float`,
- ▶ `double`, dan

► `bool`.

Kita akan membahas masing-masing secara bertahap.

3.7.1 `int`

Tipe `int` digunakan untuk bilangan bulat.

Contoh:

```
int number_of_sensors = 8;
int iteration = 0;
int sample_count = 100;
```

Bilangan seperti `-5`, `0`, `27` dapat direpresentasikan menggunakan tipe integer selama masih berada dalam rentang tipe tersebut.

Ukuran `int` tidak harus sama pada semua sistem.

Untuk memeriksanya kita dapat menggunakan `sizeof` misalnya,

```
#include <stdio.h>

int main(void)
{
    printf("sizeof(int) = %zu byte\n", sizeof(int));

    return 0;
}
```

Pada banyak komputer modern,

```
sizeof(int) = 4 byte
```

tetapi program sebaiknya tidak menganggap hal tersebut sebagai aturan universal tanpa alasan.

3.7.2 Batas integer

Pustaka `#include <limits.h>` menyediakan informasi seperti

```
INT_MIN
INT_MAX
```

Contoh:

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("INT_MIN = %d\n", INT_MIN);
    printf("INT_MAX = %d\n", INT_MAX);
}
```

```
    return 0;
}
```

Pada banyak sistem dengan `int` 32-bit, hasilnya adalah sekitar

$$-2^{31} \leq x \leq 2^{31} - 1.$$

Konsep ini berkaitan langsung dengan representasi integer yang telah dibahas pada Kuliah 1.

3.7.3 float

Tipe `float` digunakan untuk bilangan *floating-point* dengan presisi terbatas.

Contoh:

```
float temperature = 25.5f;
```

Huruf `f` pada `25.5f` menunjukkan bahwa literal tersebut bertipe `float`.

Tanpa `f`, literal seperti `25.5` secara default bertipe `double`.

3.7.4 double

Untuk banyak perhitungan sains dan rekayasa, kita akan lebih sering menggunakan `double` daripada `float`.

Contoh:

```
double pi = 3.141592653589793;
double pressure = 101325.0;
double energy = 1.23e-6;
```

`double` biasanya memiliki presisi lebih tinggi daripada `float`.

Secara umum, untuk komputasi numerik pada mata kuliah ini, `double` akan menjadi pilihan bawaan untuk besaran real kecuali ada alasan khusus menggunakan `float`.

3.7.5 Presisi float dan double

Kita dapat menggunakan `#include <float.h>` untuk melihat karakteristik tipe *floating-point*.

Contoh:

```
#include <stdio.h>
#include <float.h>

int main(void)
{
    printf("FLT_DIG = %d\n", FLT_DIG);
    printf("DBL_DIG = %d\n", DBL_DIG);
}
```

```
    return 0;  
}
```

`FLT_DIG` dan `DBL_DIG` memberikan gambaran jumlah digit desimal signifikan yang dapat direpresentasikan secara andal.

Pada banyak sistem,

```
float    ≈ 6-7 digit desimal signifikan  
double  ≈ 15-16 digit desimal signifikan
```

Nilai pastinya bergantung pada implementasi.

Konsep ini berkaitan dengan IEEE 754 yang dibahas sebelumnya.

3.7.6 char

Tipe `char` digunakan untuk menyimpan satu karakter.

Contoh:

```
char grade = 'A';
```

Perhatikan penggunaan tanda petik tunggal, `'A'` untuk karakter.

Sementara itu, `"A"` menggunakan tanda petik ganda dan merepresentasikan *string*.

Kita akan membahas *string* lebih lanjut ketika mempelajari larik karakter.

Karakter pada akhirnya direpresentasikan menggunakan nilai numerik.

Karena itu, `char` juga memiliki hubungan langsung dengan representasi data yang dibahas pada Kuliah 1.

3.7.7 bool

Secara logika, kita sering membutuhkan nilai

```
benar  
salah
```

Dalam C17, kita dapat menggunakan `#include <stdbool.h>` dan tipe `bool`.

Contoh:

```
#include <stdbool.h>  
  
bool sensor_active = true;  
bool alarm = false;
```

Nilai Boolean yang tersedia adalah

```
true  
false
```

Tipe Boolean akan menjadi sangat penting ketika kita mempelajari percabangan dan pengulangan.

3.8 Melihat ukuran tipe data

Kita dapat menggunakan `sizeof` untuk memeriksa jumlah byte yang digunakan suatu tipe.

```
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    printf("char   : %zu byte\n", sizeof(char));
    printf("int    : %zu byte\n", sizeof(int));
    printf("float   : %zu byte\n", sizeof(float));
    printf("double  : %zu byte\n", sizeof(double));
    printf("bool    : %zu byte\n", sizeof(bool));

    return 0;
}
```

Pada suatu sistem, keluaran mungkin terlihat seperti

```
char   : 1 byte
int    : 4 byte
float  : 4 byte
double : 8 byte
bool   : 1 byte
```

Namun, penting untuk memahami bahwa standar C tidak menetapkan ukuran yang sama untuk semua tipe pada semua arsitektur.

Yang lebih penting adalah memahami konsep:

tipe data → representasi → rentang dan presisi.

3.9 Konstanta

Tidak semua nilai dalam program seharusnya berubah.

Misalkan kita menggunakan percepatan gravitasi pendekatan

$$g = 9.81 \text{ m/s}^2.$$

Kita dapat menuliskannya sebagai

```
const double gravity = 9.81;
```

Kata `const` menyatakan bahwa nilai tersebut tidak dimaksudkan untuk diubah setelah inisialisasi.

Contoh:

```
const double pi = 3.141592653589793;  
const double gas_constant = 8.314462618;
```

Menggunakan `const` membantu mendokumentasikan maksud program.

Bandingkan

```
double gas_constant = 8.314462618;
```

dengan

```
const double gas_constant = 8.314462618;
```

Versi kedua menyatakan dengan lebih jelas bahwa nilai tersebut diperlakukan sebagai konstanta.

3.10 Literal numerik

Nilai yang ditulis langsung di dalam kode disebut **literal**.

Contoh literal integer:

```
0  
10  
-25  
1000
```

Contoh literal *floating-point*:

```
3.14  
2.5  
1.0e-6  
6.022e23
```

Notasi `1.0e-6` merepresentasikan 1.0×10^{-6} .

Notasi ilmiah sangat berguna untuk besaran fisika yang sangat besar atau sangat kecil.

Contoh:

```
double planck_constant = 6.62607015e-34;  
double speed_of_light = 2.99792458e8;
```

3.11 Ekspresi aritmetika

C menyediakan beberapa operator aritmetika dasar.

Operator	Operasi
+	penjumlahan
-	pengurangan
*	perkalian
/	pembagian
%	sisa pembagian integer

Contoh:

```
double voltage = 12.0;
double current = 2.5;

double power = voltage * current;
```

Secara matematis,

$$P = VI.$$

3.11.1 Penjumlahan dan pengurangan

```
double final_temperature = initial_temperature + delta_temperature;
```

merepresentasikan

$$T_f = T_i + \Delta T.$$

3.11.2 Perkalian

Dalam matematika kita dapat menulis

$$E_k = \frac{1}{2}mv^2.$$

Dalam C, kita harus menuliskan operator perkalian secara eksplisit.

```
double energy = 0.5 * mass * velocity * velocity;
```

Tidak ada perkalian implisit seperti `mv` dalam sintaks C.

3.11.3 Pangkat

C tidak mempunyai operator pangkat `^` untuk bilangan real.

Ekspresi `v ^ 2` bukan berarti v^2 .

Operator `^` dalam C adalah operator bitwise XOR.

Untuk kuadrat sederhana, v^2 lebih baik ditulis `v * v`.

Untuk operasi pangkat yang lebih umum, pustaka matematika menyediakan fungsi `pow()` melalui `#include <math.h>`.

Contoh:

```
double y = pow(x, 3.5);
```

Namun, untuk kuadrat sederhana, `x * x` sering lebih mudah dibaca dan tidak memerlukan pemanggilan fungsi `pow`.

3.12 Pembagian integer

Salah satu sumber kesalahan umum bagi pemula adalah pembagian integer.

Perhatikan

```
int a = 5;
int b = 2;

int result = a / b;
```

Hasilnya adalah 2, bukan 2.5.

Karena kedua operand bertipe integer, C melakukan pembagian integer.

Bagian pecahan dibuang.

$$\frac{5}{2} \longrightarrow 2$$

dalam konteks pembagian integer.

Jika kita ingin hasil *floating-point*,

```
double result = 5.0 / 2.0;
```

menghasilkan 2.5.

3.12.1 Kesalahan klasik pada rata-rata

Perhatikan kode

```
int sum = 7;
int n = 2;

double mean = sum / n;
```

Mungkin kita berharap

$$\bar{x} = 3.5.$$

Namun, operasi `sum / n` dilakukan sebagai pembagian integer terlebih dahulu,

$$7/2 = 3,$$

kemudian nilai 3 dikonversi menjadi `double`.

Hasilnya menjadi 3.0.

Salah satu perbaikannya adalah

```
double mean = (double)sum / n;
```

atau menggunakan `double` sejak awal,

```
double sum = 7.0;  
int n = 2;  
  
double mean = sum / n;
```

Contoh ini menunjukkan bahwa **tipe data memengaruhi hasil perhitungan**.

3.13 Operator modulo %

Operator % menghasilkan sisa pembagian integer.

Contoh:

```
int remainder = 17 % 5;
```

menghasilkan 2 karena

$$17 = 3(5) + 2.$$

Operator modulo banyak digunakan untuk

- ▶ memeriksa bilangan genap atau ganjil;
- ▶ pola periodik;
- ▶ pengindeksan;
- ▶ pengolahan data diskrit.

Sebagai contoh,

```
int n = 8;  
  
int remainder = n % 2;
```

Jika `remainder == 0` maka `n` genap.

3.14 Prioritas operator

C memiliki aturan prioritas operator.

Sebagai contoh,

```
double result = a + b * c;
```

perkalian dilakukan lebih dahulu.

Secara matematis, $a + bc$.

Jika kita ingin $(a + b)c$, kita harus menulis

```
double result = (a + b) * c;
```

Beberapa aturan dasar yang perlu diingat:

1. ekspresi dalam tanda kurung dievaluasi terlebih dahulu;

2. perkalian, pembagian, dan modulo memiliki prioritas lebih tinggi daripada penjumlahan dan pengurangan;
3. operator dengan prioritas sama mengikuti aturan asosiasi tertentu.

Namun, jangan terlalu bergantung pada hafalan tabel prioritas.

Jika ekspresi mulai sulit dibaca, gunakan tanda kurung.

Bandingkan

```
double result = a + b * c / d - e;
```

dengan

```
double result = a + (b * c / d) - e;
```

Versi kedua lebih eksplisit.

3.15 Operator relasional

Program sering perlu membandingkan dua nilai.

C menyediakan operator relasional berikut.

Operator	Arti
<	lebih kecil
<=	lebih kecil atau sama dengan
>	lebih besar
>=	lebih besar atau sama dengan
==	sama dengan
!=	tidak sama dengan

Contoh:

```
temperature > 30.0
```

menanyakan apakah temperatur lebih besar dari 30.

Ekspresi `temperature >= 20.0` menanyakan apakah temperatur lebih besar atau sama dengan 20.

3.15.1 Jangan tertukar antara = dan ==

Ini merupakan salah satu kesalahan paling umum.

```
x = 5;
```

berarti

simpan nilai 5 ke x.

Sedangkan `x == 5` berarti

apakah nilai `x` sama dengan 5?

Operator pertama adalah assignment.

Operator kedua adalah operator perbandingan.

Perbedaannya sangat penting.

3.16 Logika Boolean

Dalam matematika dan logika, kita mengenal operasi $p \wedge q$, $p \vee q$, dan $\neg p$.

Dalam C, operator logikanya adalah

Logika	C
AND	<code>&&</code>
OR	<code> </code>
NOT	<code>!</code>

Misalkan

$$20 \leq T \leq 30.$$

Dalam C kita tidak menulis `20.0 <= temperature <= 30.0` karena ekspresi tersebut tidak memiliki makna yang sama seperti notasi matematika.

Kita harus menulis dua kondisi dan menggabungkannya dengan AND.

```
temperature >= 20.0 && temperature <= 30.0
```

Secara logika,

$$(T \geq 20) \wedge (T \leq 30).$$

3.16.1 Operator AND

Misalkan suatu sensor dianggap berada dalam rentang aman jika

$$20 \leq T \leq 30.$$

Dalam C,

```
bool safe =  
    temperature >= 20.0 &&  
    temperature <= 30.0;
```

Nilai `safe` akan menjadi `true` jika kedua kondisi benar.

3.16.2 Operator OR

Misalkan alarm aktif jika

$$T < 10$$

atau

$$T > 50.$$

Dalam C,

```
bool alarm =
    temperature < 10.0 ||
    temperature > 50.0;
```

3.16.3 Operator NOT

Jika

```
bool sensor_active = true;
```

maka `!sensor_active` berarti

sensor tidak aktif.

3.17 Tabel kebenaran

Untuk dua nilai Boolean p dan q , operasi AND dapat ditulis

p	q	$p \wedge q$
false	false	false
false	true	false
true	false	false
true	true	true

Operasi OR:

p	q	$p \vee q$
false	false	false
false	true	true
true	false	true
true	true	true

Operasi NOT:

p	$\neg p$
false	true
true	false

Konsep ini akan menjadi dasar utama ketika kita mempelajari percabangan pada kuliah berikutnya.

3.18 Konversi tipe data

Dalam suatu ekspresi, kita kadang mencampur beberapa tipe data.

Contoh:

```
int n = 4;
double sum = 10.0;

double mean = sum / n;
```

C akan melakukan konversi sehingga `n` dapat digunakan dalam operasi dengan `double`. Konversi yang dilakukan otomatis disebut **implicit conversion**.

3.18.1 Konversi eksplisit

Kita juga dapat meminta konversi secara eksplisit menggunakan **cast**.

Contoh:

```
int sum = 7;
int n = 2;

double mean = (double)sum / n;
```

Bagian `(double)sum` mengubah nilai `sum` menjadi `double` sebelum pembagian.

Dengan demikian,

$$\frac{7.0}{2} = 3.5.$$

3.18.2 Konversi dapat kehilangan informasi

Perhatikan

```
double x = 3.9;
int y = (int)x;
```

Nilai `y` menjadi 3.

Bagian pecahan tidak disimpan.

Konversi dari tipe dengan informasi lebih banyak ke tipe dengan informasi lebih sedikit dapat kehilangan informasi.

Karena itu, `cast` tidak boleh digunakan hanya untuk menghilangkan peringatan *compiler* tanpa memahami konsekuensinya.

3.19 Input dan output

Program yang hanya menggunakan nilai tetap memiliki kegunaan terbatas.

Kita ingin program dapat menerima masukan dari pengguna dan menampilkan hasil.

Pustaka `#include <stdio.h>` menyediakan fungsi utama untuk kebutuhan awal ini:

```
printf()
scanf()
```

3.19.1 printf

Fungsi `printf()` digunakan untuk menampilkan keluaran.

Contoh:

```
printf("Hello!\n");
```

Karakter `\n` menyatakan baris baru.

3.19.2 Menampilkan integer

```
int n = 25;

printf("n = %d\n", n);
```

`%d` digunakan untuk menampilkan nilai bertipe `int`.

Keluaran:

```
n = 25
```

3.19.3 Menampilkan bilangan real

```
double temperature = 29.875;

printf("Temperature = %f C\n", temperature);
```

menghasilkan sesuatu seperti

```
Temperature = 29.875000 C
```

Kita dapat mengatur jumlah digit setelah titik desimal.

```
printf("Temperature = %.2f C\n", temperature);
```

menghasilkan

```
Temperature = 29.88 C
```

Format `%.2f` berarti menampilkan dua digit setelah titik desimal.

3.19.4 Notasi ilmiah

Untuk nilai sangat besar atau kecil,

```
double h = 6.62607015e-34;
printf("h = %e J s\n", h);
```

dapat menghasilkan

```
h = 6.626070e-34 J s
```

Format `%e` menampilkan notasi ilmiah.

3.19.5 Beberapa format yang sering digunakan

Tipe	printf
int	%d
char	%c
double	%f, %e, %g
string	%s

Untuk `printf`, argumen bertipe `float` dipromosikan menjadi `double`, sehingga `%f` digunakan untuk keduanya.

3.20 Membaca masukan dengan `scanf`

Fungsi `scanf()` digunakan untuk membaca masukan.

Contoh integer:

```
int n;
scanf("%d", &n);
```

Contoh `double`:

```
double temperature;
scanf("%lf", &temperature);
```

Perhatikan tanda `&` di depan nama variabel.

Pada tahap ini kita cukup memahami bahwa `scanf` membutuhkan **alamat** variabel agar dapat menyimpan nilai yang dibaca.

Konsep alamat dan operator `&` akan dibahas lebih rinci ketika kita mempelajari *pointer*.

3.20.1 `%f` dan `%lf` pada `scanf`

Perbedaan berikut sangat penting.

Untuk membaca `float`:

```
float x;  
  
scanf("%f", &x);
```

Untuk membaca `double`:

```
double x;  
  
scanf("%lf", &x);
```

Jadi pada `scanf`,

```
%f  → float *  
%lf → double *
```

Hal ini berbeda dari `printf`, di mana `%f` digunakan untuk menampilkan `double`.

3.21 Contoh input-output sederhana

Program berikut membaca massa dan kecepatan, kemudian menghitung energi kinetik.

```
#include <stdio.h>  
  
int main(void)  
{  
    double mass;  
    double velocity;  
  
    printf("Mass (kg): ");  
    scanf("%lf", &mass);  
  
    printf("Velocity (m/s): ");  
    scanf("%lf", &velocity);  
  
    double energy =  
        0.5 * mass * velocity * velocity;  
  
    printf("Kinetic energy = %.3f J\n", energy);  
  
    return 0;  
}
```

Jika masukan adalah

```
Mass (kg): 2  
Velocity (m/s): 3
```

keluarannya adalah

```
Kinetic energy = 9.000 J
```

Sebelum menjalankan program, kita dapat memprediksi

$$E_k = \frac{1}{2}(2)(3)^2 = 9 \text{ J.}$$

Kebiasaan membandingkan prediksi manual dengan keluaran program harus terus dipertahankan.

3.22 Memeriksa keberhasilan `scanf`

`scanf` mengembalikan jumlah item yang berhasil dibaca.

Contoh:

```
double mass;  
  
int result = scanf("%lf", &mass);
```

Jika satu nilai `double` berhasil dibaca, `result == 1`.

Kita dapat memanfaatkannya untuk pemeriksaan dasar.

```
#include <stdio.h>  
  
int main(void)  
{  
    double mass;  
  
    printf("Mass (kg): ");  
  
    if (scanf("%lf", &mass) != 1)  
    {  
        printf("Invalid input.\n");  
        return 1;  
    }  
  
    printf("Mass = %.3f kg\n", mass);  
  
    return 0;  
}
```

Struktur `if` akan dibahas secara sistematis pada kuliah berikutnya.

Untuk sekarang, contoh tersebut menunjukkan bahwa program teknik sebaiknya tidak selalu menganggap masukan pengguna benar.

3.23 Studi kasus 1: energi kinetik

Kita mulai dari model

$$E_k = \frac{1}{2}mv^2.$$

3.23.1 Spesifikasi

Masukan:

$$m, \quad v.$$

Keluaran:

$$E_k.$$

Asumsi:

$$m \geq 0.$$

3.23.2 Pseudocode

```
INPUT mass
INPUT velocity
energy  $\leftarrow$  0.5 · mass · velocity · velocity
OUTPUT energy
```

3.23.3 Implementasi C

```
#include <stdio.h>

int main(void)
{
    double mass;
    double velocity;

    printf("Mass (kg): ");
    scanf("%lf", &mass);

    printf("Velocity (m/s): ");
    scanf("%lf", &velocity);

    double energy =
        0.5 * mass * velocity * velocity;

    printf("Kinetic energy = %.6f J\n", energy);

    return 0;
}
```

3.23.4 Pemeriksaan

Untuk

$$m = 2.0 \text{ kg}$$

dan

$$v = 3.0 \text{ m/s},$$

hasil yang diharapkan adalah

$$E_k = 9.0 \text{ J}.$$

Jika program menghasilkan nilai yang sangat berbeda, kita harus memeriksa

- ▶ input;
- ▶ satuan;
- ▶ ekspresi;
- ▶ tipe data;
- ▶ format output.

3.24 Studi kasus 2: Hukum Ohm

Hukum Ohm diberikan oleh

$$V = IR.$$

Jika I dan R diketahui, kita dapat menghitung V .

3.24.1 Pseudocode

```
INPUT current
INPUT resistance
voltage ← current · resistance
OUTPUT voltage
```

3.24.2 Implementasi C

```
#include <stdio.h>

int main(void)
{
    double current;
    double resistance;

    printf("Current (A): ");
    scanf("%lf", &current);

    printf("Resistance (ohm): ");
    scanf("%lf", &resistance);

    double voltage =
        current * resistance;

    printf("Voltage = %.3f V\n", voltage);

    return 0;
}
```

Untuk

$$I = 2.5 \text{ A}$$

dan

$$R = 10 \, \Omega,$$

kita memperoleh

$$V = (2.5)(10) = 25 \text{ V.}$$

3.25 Studi kasus 3: konversi Celsius ke Kelvin

Hubungan antara Celsius dan Kelvin adalah

$$T_K = T_C + 273.15.$$

Program:

```
#include <stdio.h>

int main(void)
{
    double temperature_c;

    printf("Temperature (C): ");
    scanf("%lf", &temperature_c);

    double temperature_k =
        temperature_c + 273.15;

    printf("Temperature = %.2f K\n", temperature_k);

    return 0;
}
```

Untuk

$$T_C = 25^\circ\text{C},$$

kita memperoleh

$$T_K = 298.15 \text{ K.}$$

Contoh ini sederhana, tetapi menunjukkan pentingnya nama variabel yang menyertakan informasi satuan.

Bandingkan

```
double t;
double x;
```

dengan

```
double temperature_c;
double temperature_k;
```

Versi kedua jauh lebih aman untuk program teknik.

3.26 Studi kasus 4: persamaan gas ideal

Persamaan gas ideal adalah

$$PV = nRT.$$

Jika kita ingin menghitung tekanan,

$$P = \frac{nRT}{V}.$$

Gunakan konstanta gas

$$R = 8.314462618 \text{ J mol}^{-1}\text{K}^{-1}.$$

Implementasi:

```
#include <stdio.h>

int main(void)
{
    const double gas_constant = 8.314462618;

    double amount;
    double temperature;
    double volume;

    printf("Amount of substance (mol): ");
    scanf("%lf", &amount);

    printf("Temperature (K): ");
    scanf("%lf", &temperature);

    printf("Volume (m^3): ");
    scanf("%lf", &volume);

    double pressure =
        amount * gas_constant * temperature / volume;

    printf("Pressure = %.6e Pa\n", pressure);

    return 0;
}
```

Misalkan

$$n = 1 \text{ mol},$$

$$T = 300 \text{ K},$$

dan

$$V = 0.024 \text{ m}^3.$$

Secara manual,

$$P = \frac{(1)(8.314462618)(300)}{0.024}.$$

Nilainya sekitar

$$P \approx 1.04 \times 10^5 \text{ Pa}.$$

Nilai tersebut masuk akal karena berada pada orde tekanan atmosfer.

Di sini kita melihat pentingnya **orde besaran** sebagai pemeriksaan fisik.

3.27 Studi kasus 5: energi foton

Energi foton diberikan oleh

$$E = hf,$$

dengan

$$h = 6.62607015 \times 10^{-34} \text{ J s.}$$

Program:

```
#include <stdio.h>

int main(void)
{
    const double planck_constant =
        6.62607015e-34;

    double frequency;

    printf("Frequency (Hz): ");
    scanf("%lf", &frequency);

    double energy =
        planck_constant * frequency;

    printf("Photon energy = %.6e J\n", energy);

    return 0;
}
```

Jika

$$f = 5.0 \times 10^{14} \text{ Hz,}$$

maka

$$E = (6.62607015 \times 10^{-34})(5.0 \times 10^{14}),$$

sehingga

$$E \approx 3.31 \times 10^{-19} \text{ J.}$$

Contoh ini menunjukkan mengapa notasi ilmiah sangat berguna dalam program sains.

3.28 Ekspresi gabungan

Program teknik sering menggunakan ekspresi yang lebih panjang.

Misalkan kita menghitung posisi

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2.$$

Dalam C,

```
double position =  
    x0  
    + v0 * time  
    + 0.5 * acceleration * time * time;
```

Menulis ekspresi dalam beberapa baris diperbolehkan.

C tidak menganggap baris baru sebagai akhir pernyataan.

Akhir pernyataan ditentukan oleh titik koma.

Karena itu,

```
double position =  
    x0  
    + v0 * time  
    + 0.5 * acceleration * time * time;
```

dan

```
double position = x0 + v0 * time + 0.5 * acceleration * time * time;
```

memiliki makna yang sama.

Versi pertama mungkin lebih mudah dibaca untuk ekspresi panjang.

3.29 Urutan evaluasi dan penggunaan variabel antara

Kadang-kadang lebih baik memecah ekspresi menjadi beberapa langkah.

Misalkan

$$E_k = \frac{1}{2}mv^2.$$

Alih-alih

```
double energy =  
    0.5 * mass * velocity * velocity;
```

kita dapat menulis

```
double velocity_squared =  
    velocity * velocity;  
  
double energy =  
    0.5 * mass * velocity_squared;
```

Versi kedua dapat lebih mudah diperiksa ketika perhitungan menjadi rumit.

Penggunaan variabel antara juga memudahkan *debugging*.

Namun, terlalu banyak variabel yang tidak perlu juga dapat membuat program panjang.

Kita perlu mencari keseimbangan antara

- ▶ keterbacaan;
- ▶ kesederhanaan;
- ▶ kemudahan pemeriksaan.

3.30 Perbandingan bilangan *floating-point*

Pada Kuliah 1 kita telah melihat bahwa bilangan seperti 0.1 tidak selalu dapat direpresentasikan secara eksak dalam biner.

Karena itu, perbandingan langsung bilangan *floating-point* perlu dilakukan dengan hati-hati.

Misalkan secara matematis kita mengharapkan

$$0.1 + 0.2 = 0.3.$$

Namun representasi komputer dapat memberikan nilai yang sangat sedikit berbeda.

Daripada selalu menggunakan kesamaan eksak, $x == y$ dalam komputasi numerik kita sering memeriksa

$$|x - y| < \varepsilon.$$

Dalam C, dengan pustaka matematika, `#include <math.h>` kita dapat menulis `fabs(x - y) < tolerance`.

Sebagai contoh,

```
#include <stdio.h>
#include <math.h>
#include <stdbool.h>

int main(void)
{
    double x = 0.1 + 0.2;
    double y = 0.3;

    const double tolerance = 1e-12;

    bool approximately_equal =
        fabs(x - y) < tolerance;

    printf("x = %.17f\n", x);
    printf("y = %.17f\n", y);
    printf("approximately equal = %d\n",
        approximately_equal);

    return 0;
}
```

Pada tahap ini kita belum perlu mendalami pemilihan toleransi.

Yang perlu dipahami adalah bahwa tipe data *floating-point* memiliki konsekuensi numerik.

3.31 Galat kompilasi, peringatan, dan galat logika

Saat belajar C, kita akan sering bertemu tiga kategori masalah.

3.31.1 Galat sintaksis

Contoh:

```
double mass = 2.0
```

Titik koma hilang.

Compiler dapat mendeteksi masalah tersebut.

3.31.2 Peringatan

Contoh:

```
int x = 3.7;
```

Konversi dari bilangan real ke integer dapat kehilangan informasi.

Tergantung *compiler* dan opsi yang digunakan, kita mungkin memperoleh peringatan.

3.31.3 Galat logika

Contoh:

```
double energy =  
    0.5 * mass * velocity;
```

Program mungkin dapat dikompilasi tanpa masalah.

Namun, model yang seharusnya

$$E_k = \frac{1}{2}mv^2$$

tidak diimplementasikan dengan benar.

Compiler tidak mengetahui fisika.

Karena itu, galat logika harus ditemukan melalui

- ▶ pengujian;
- ▶ perhitungan manual;
- ▶ pemeriksaan satuan;
- ▶ pemeriksaan orde besaran;
- ▶ penalaran terhadap algoritma.

3.32 Compiler warning sebagai bagian dari proses belajar

Gunakan

```
gcc -Wall -Wextra program.c -o program
```

dan baca setiap pesan dengan teliti.

Sebagai kebiasaan awal:

jangan abaikan warning tanpa memahami penyebabnya

Peringatan dapat menunjukkan

- ▶ variabel yang tidak digunakan;
- ▶ konversi tipe;
- ▶ format input-output yang tidak sesuai;
- ▶ kemungkinan kesalahan logika;
- ▶ bagian program yang mencurigakan.

Tujuan kita bukan hanya membuat *compiler* diam, tetapi memahami mengapa suatu peringatan muncul.

3.33 Studi kasus terpadu: daya pada resistor

Misalkan kita ingin membuat program yang membaca hambatan dan arus, kemudian menghitung

$$V = IR$$

dan

$$P = VI.$$

Dengan substitusi,

$$P = I^2 R.$$

3.33.1 Spesifikasi

Masukan:

$$I, \quad R.$$

Keluaran:

$$V, \quad P.$$

Asumsi:

$$R \geq 0.$$

3.33.2 Pseudocode

```

INPUT current
INPUT resistance
voltage ← current · resistance
power ← voltage · current
OUTPUT voltage
OUTPUT power
  
```

3.33.3 Implementasi C

```
#include <stdio.h>

int main(void)
{
    double current;
    double resistance;

    printf("Current (A): ");
    scanf("%lf", &current);

    printf("Resistance (ohm): ");
    scanf("%lf", &resistance);

    double voltage =
        current * resistance;

    double power =
        voltage * current;

    printf("\nVoltage = %.3f V\n", voltage);
    printf("Power = %.3f W\n", power);

    return 0;
}
```

3.33.4 Tracing manual

Gunakan

$$I = 2.5 \text{ A},$$

$$R = 10 \text{ } \Omega.$$

Maka

$$V = IR = (2.5)(10) = 25 \text{ V}.$$

Selanjutnya,

$$P = VI = (25)(2.5) = 62.5 \text{ W}.$$

Program seharusnya menghasilkan nilai mendekati

```
Voltage = 25.000 V
Power   = 62.500 W
```

3.33.5 Pemeriksaan dimensi

Untuk tegangan,

$$[I][R] = \text{A} \cdot \Omega = \text{V}.$$

Untuk daya,

$$[V][I] = \text{V} \cdot \text{A} = \text{W}.$$

Pemeriksaan satuan merupakan salah satu alat *debugging* yang sangat kuat dalam komputasi fisika.

3.34 Studi kasus terpadu: gerak dengan percepatan konstan

Model:

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2,$$

dan

$$v(t) = v_0 + a t.$$

3.34.1 Pseudocode

```
INPUT  $x_0$ 
INPUT  $v_0$ 
INPUT  $a$ 
INPUT  $t$ 
 $position \leftarrow x_0 + v_0 \cdot t + 0.5 \cdot a \cdot t \cdot t$ 
 $velocity \leftarrow v_0 + a \cdot t$ 
OUTPUT  $position$ 
OUTPUT  $velocity$ 
```

3.34.2 Implementasi

```
#include <stdio.h>

int main(void)
{
    double x0;
    double v0;
    double acceleration;
    double time;

    printf("Initial position (m): ");
    scanf("%lf", &x0);

    printf("Initial velocity (m/s): ");
    scanf("%lf", &v0);

    printf("Acceleration (m/s^2): ");
    scanf("%lf", &acceleration);

    printf("Time (s): ");
    scanf("%lf", &time);

    double position =
        x0
        + v0 * time
        + 0.5 * acceleration * time * time;
```

```

double velocity =
    v0 + acceleration * time;

printf("\nPosition = %.3f m\n", position);
printf("Velocity = %.3f m/s\n", velocity);

return 0;
}

```

Untuk

$$\begin{aligned}
 x_0 &= 2 \text{ m}, \\
 v_0 &= 5 \text{ m/s}, \\
 a &= 3 \text{ m/s}^2,
 \end{aligned}$$

dan

$$t = 4 \text{ s},$$

kita memperoleh

$$x = 2 + (5)(4) + \frac{1}{2}(3)(4)^2 = 46 \text{ m},$$

serta

$$v = 5 + (3)(4) = 17 \text{ m/s}.$$

Keluaran program dapat dibandingkan langsung dengan hasil tersebut.

3.35 Kesalahan umum pada tahap awal pemrograman C

3.35.1 Melupakan titik koma

Salah:

```
double mass = 2.0
```

Benar:

```
double mass = 2.0;
```

3.35.2 Menggunakan variabel sebelum diberi nilai

Salah:

```
double x;
double y = x + 1.0;
```

Lebih baik:

```
double x = 0.0;
double y = x + 1.0;
```

3.35.3 Salah memilih tipe data

Contoh:

```
int temperature = 25.75;
```

Nilai pecahan tidak dapat dipertahankan sebagai `int`.

Gunakan

```
double temperature = 25.75;
```

3.35.4 Pembagian integer yang tidak disadari

```
double x = 1 / 2;
```

memberikan 0.0 karena 1/2 dihitung sebagai pembagian integer terlebih dahulu.

Gunakan

```
double x = 1.0 / 2.0;
```

atau

```
double x = 0.5;
```

3.35.5 Salah menulis format `scanf`

Jika variabel bertipe `double`,

```
double x;
```

gunakan

```
scanf("%lf", &x);
```

bukan

```
scanf("%f", &x);
```

3.35.6 Melupakan & pada `scanf`

Salah:

```
scanf("%lf", temperature);
```

Benar:

```
scanf("%lf", &temperature);
```

3.35.7 Menggunakan `=` ketika maksudnya membandingkan

Assignment:

```
x = 5;
```

Perbandingan:

```
x == 5
```

Keduanya memiliki makna berbeda.

3.35.8 Menganggap $^$ sebagai pangkat

Salah:

```
energy = 0.5 * mass * velocity ^ 2;
```

Benar untuk kuadrat sederhana:

```
energy =  
    0.5 * mass * velocity * velocity;
```

3.36 Kebiasaan pemrograman yang baik

Sejak awal, biasakan beberapa praktik berikut.

3.36.1 Gunakan nama variabel yang informatif

Lebih baik:

```
double temperature;  
double pressure;  
double velocity;
```

daripada

```
double a;  
double b;  
double c;
```

kecuali konteksnya memang sangat sederhana.

3.36.2 Nyatakan satuan

Contoh:

```
double pressure_pa;  
double temperature_k;
```

atau gunakan komentar.

```
double velocity = 20.0; // m/s
```

3.36.3 Gunakan `const` untuk konstanta

```
const double gravity = 9.81;
```

3.36.4 Gunakan indentasi konsisten

```
int main(void)
{
    double x = 1.0;

    printf("%f\n", x);

    return 0;
}
```

Indentasi membantu menunjukkan struktur program.

3.36.5 Batasi panjang ekspresi

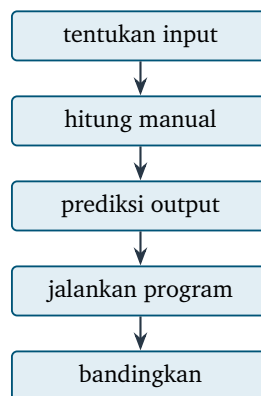
Jika ekspresi sulit dibaca, pecah menjadi variabel antara.

3.36.6 Kompilasi dengan warning

```
gcc -Wall -Wextra program.c -o program
```

3.36.7 Prediksi sebelum menjalankan

Untuk setiap program sederhana, lakukan



Kebiasaan ini jauh lebih efektif daripada hanya menekan tombol *Run* berulang kali.

3.37 Cek pemahaman

Cobalah menjawab pertanyaan berikut tanpa melihat kembali catatan.

- ▶ Apa peran fungsi `main` dalam program C?
- ▶ Mengapa `#include <stdio.h>` diperlukan ketika menggunakan `printf` dan `scanf`?
- ▶ Apa perbedaan deklarasi, inisialisasi, dan assignment?
- ▶ Mengapa variabel sebaiknya tidak digunakan sebelum diberi nilai?
- ▶ Apa perbedaan `int`, `float`, dan `double`?
- ▶ Mengapa `double` sering lebih sesuai daripada `float` untuk komputasi ilmiah?
- ▶ Apa fungsi `const`?
- ▶ Apa perbedaan antara literal `2`, `2.0`, dan `2.0f`?
- ▶ Mengapa `5 / 2` berbeda dari `5.0 / 2.0`?
- ▶ Apa fungsi operator `%`?
- ▶ Apa perbedaan antara `=` dan `==`?
- ▶ Bagaimana menuliskan kondisi matematis

$$20 \leq T \leq 30$$

dalam C?

- ▶ Apa arti operator `&&`, `||`, dan `!`?
- ▶ Mengapa `v ^ 2` bukan cara yang benar untuk menulis v^2 dalam C?
- ▶ Apa perbedaan *implicit conversion* dan *explicit cast*?
- ▶ Mengapa konversi dari `double` ke `int` dapat kehilangan informasi?
- ▶ Format apa yang digunakan `scanf` untuk membaca `double`?
- ▶ Mengapa `scanf` membutuhkan `&` di depan variabel?
- ▶ Apa manfaat `-Wall` dan `-Wextra`?
- ▶ Mengapa program yang berhasil dikompilasi belum tentu benar secara fisika?

3.38 Latihan

3.38.1 Struktur program C

Tuliskan program C yang hanya menampilkan

Algoritma dan Pemrograman
Teknik Fisika

Gunakan `printf()` dan pastikan setiap teks tampil pada baris yang berbeda.

Kompilasi menggunakan

```
gcc -Wall -Wextra program.c -o program
```

Pastikan tidak muncul peringatan.

3.38.2 Variabel dan tipe data

Buat variabel untuk merepresentasikan

- ▶ jumlah sensor;
- ▶ temperatur;
- ▶ tekanan;
- ▶ karakter status;
- ▶ kondisi alarm aktif atau tidak.

Pilih tipe data yang sesuai dari

```
int
double
char
bool
```

Jelaskan alasan pemilihan tipe masing-masing.

3.38.3 Ukuran tipe data

Buat program yang menampilkan

```
sizeof(char)
sizeof(int)
sizeof(float)
sizeof(double)
sizeof(bool)
```

Jalankan pada komputer masing-masing.

Bandingkan hasilnya dengan teman yang menggunakan sistem operasi atau arsitektur berbeda jika tersedia.

Apakah semua hasil harus sama?

3.38.4 Rentang integer

Gunakan `#include <limits.h>` untuk menampilkan

```
INT_MIN
INT_MAX
```

Bandingkan dengan prediksi berdasarkan jumlah bit `int` pada sistem operasi dan lingkungan pemrograman masing-masing.

3.38.5 Pembagian integer

Prediksi hasil program berikut sebelum menjalankannya.

```
#include <stdio.h>

int main(void)
{
    int a = 7;
```

```
int b = 2;

double x = a / b;
double y = (double)a / b;

printf("x = %f\n", x);
printf("y = %f\n", y);

return 0;
}
```

Jelaskan mengapa x dan y berbeda.

3.38.6 Konversi temperatur

Buat program yang membaca temperatur Celsius dan menghitung Kelvin menggunakan

$$T_K = T_C + 273.15.$$

Gunakan `double`.

Uji menggunakan

$$T_C = 0^\circ\text{C},$$

$$T_C = 25^\circ\text{C},$$

dan

$$T_C = 100^\circ\text{C}.$$

Hitung hasil manual terlebih dahulu.

3.38.7 Hukum Ohm

Buat program yang membaca I dan R , kemudian menghitung

$$V = IR$$

dan

$$P = I^2R.$$

Tampilkan hasil dengan tiga digit setelah titik desimal.

Gunakan

$$I = 2.5 \text{ A},$$

dan

$$R = 10 \, \Omega$$

sebagai kasus uji.

3.38.8 Gerak lurus

Buat program yang membaca

$$x_0, \quad v_0, \quad a, \quad t,$$

kemudian menghitung

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

dan

$$v(t) = v_0 + a t.$$

Gunakan kasus uji

$$x_0 = 2 \text{ m},$$

$$v_0 = 5 \text{ m/s},$$

$$a = 3 \text{ m/s}^2,$$

$$t = 4 \text{ s}.$$

Prediksi keluaran sebelum menjalankan program.

3.38.9 Persamaan gas ideal

Gunakan

$$P = \frac{nRT}{V}$$

dengan

$$R = 8.314462618 \text{ J mol}^{-1} \text{K}^{-1}.$$

Buat program yang membaca n , T , V dan menampilkan tekanan dalam pascal.

Gunakan `const double` untuk R .

Uji menggunakan

$$n = 1 \text{ mol},$$

$$T = 300 \text{ K},$$

$$V = 0.024 \text{ m}^3.$$

Periksa apakah orde besaran hasil mendekati tekanan atmosfer.

3.38.10 Energi foton

Energi foton diberikan oleh

$$E = hf.$$

Gunakan

$$h = 6.62607015 \times 10^{-34} \text{ J s}.$$

Buat program yang membaca frekuensi dan menampilkan energi menggunakan format notasi ilmiah.

Uji menggunakan

$$f = 5.0 \times 10^{14} \text{ Hz}.$$

3.38.11 Ekspresi Boolean

Misalkan temperatur sistem adalah `temperature`.

Tuliskan ekspresi Boolean C untuk kondisi berikut.

1. Temperatur lebih besar dari 30°C.

2. Temperatur berada pada rentang

$$20^{\circ}\text{C} \leq T \leq 30^{\circ}\text{C}.$$

3. Temperatur berada di luar rentang aman

$$10^{\circ}\text{C} \leq T \leq 50^{\circ}\text{C}.$$

4. Sensor aktif dan temperatur lebih besar dari 40°C.

5. Sensor tidak aktif.

3.38.12 Mencari kesalahan

Perhatikan program berikut.

```
#include <stdio.h>

int main(void)
{
    int mass = 2.5;
    double velocity;

    printf("Velocity: ");
    scanf("%f", velocity);

    double energy =
        1 / 2 * mass * velocity ^ 2;

    printf("Energy = %d\n", energy)

    return 0;
}
```

Program tersebut memiliki beberapa kesalahan konsep dan sintaks.

Identifikasi sebanyak mungkin masalah, kemudian tulis versi yang benar.

3.38.13 Pemeriksaan input sederhana

Modifikasi program energi kinetik sehingga program memeriksa apakah `scanf` berhasil membaca nilai massa dan kecepatan.

Jika pembacaan gagal, tampilkan pesan `Invalid input`. dan akhiri program dengan

```
return 1;
```

3.38.14 Eksperimen presisi *floating-point*

Buat program

```
#include <stdio.h>

int main(void)
{
    double x = 0.1 + 0.2;

    printf("%.17f\n", x);

    return 0;
}
```

Jalankan program dan amati hasilnya. Bandingkan dengan 0.3. Jelaskan hubungan hasil eksperimen dengan pembahasan representasi *floating-point* pada Kuliah 1.

3.39 Rangkuman

- ▶ Pada kuliah ini kita mulai menerjemahkan algoritma menjadi program C.
- ▶ Struktur dasar program C dapat ditulis sebagai

```
#include <stdio.h>

int main(void)
{
    /* program */

    return 0;
}
```

- ▶ Program C dikompilasi menjadi *executable* sebelum dijalankan.
- ▶ Variabel merupakan lokasi penyimpanan bernama yang mempunyai tipe tertentu.
- ▶ Deklarasi menentukan nama dan tipe variabel, sedangkan inisialisasi memberikan nilai awal.
- ▶ *Assignment* menggunakan operator `=` yang berbeda dari perbandingan kesamaan `==`
- ▶ Tipe data dasar yang penting pada tahap ini meliputi
 - `char`,
 - `int`,
 - `float`,
 - `double`, dan
 - `bool`
- ▶ Untuk komputasi ilmiah dasar, `double` akan sering digunakan karena menyediakan presisi lebih tinggi daripada `float`.

- ▶ Tipe data menentukan representasi, rentang nilai, dan presisi.
- ▶ Nilai yang tidak dimaksudkan berubah dapat dideklarasikan menggunakan `const`
- ▶ Operator aritmetika dasar meliputi `+` `-` `*` `/` `%`
- ▶ Pembagian bilangan bulat (*integer*) harus diperhatikan karena

$$5/2 = 2$$

jika kedua *operand* bertipe *integer*.

- ▶ C tidak menggunakan `^` sebagai operator pangkat. Untuk kuadrat sederhana, gunakan `x * x`
- ▶ Operator relasional meliputi `<` `<=` `>` `>=` `==` `!=`
- ▶ Operator logika meliputi `&&` `||` `!` yang masing-masing merepresentasikan AND, OR, dan NOT.
- ▶ Kondisi matematika seperti

$$20 \leq T \leq 30$$

ditulis dalam C sebagai

```
temperature >= 20.0 &&
temperature <= 30.0
```

- ▶ Konversi tipe dapat terjadi secara implisit atau dilakukan secara eksplisit menggunakan `cast`.
- ▶ `printf` digunakan untuk keluaran, sedangkan `scanf` digunakan untuk masukan.
- ▶ Untuk `scanf`,

```
%d   → int
%f   → float
%lf  → double
```

- ▶ `scanf` memerlukan alamat variabel sehingga kita menggunakan operator `&`.
- ▶ Program teknik sebaiknya dikompilasi dengan peringatan aktif, misalnya

```
gcc -Wall -Wextra program.c -o program
```

- ▶ Program yang berhasil dikompilasi belum tentu benar. Hasil tetap perlu diperiksa menggunakan algoritma, perhitungan manual, satuan, orde besaran, dan penalaran fisika.
- ▶ Pada kuliah berikutnya kita akan menggunakan ekspresi Boolean yang telah dipelajari untuk membangun aliran kontrol program melalui percabangan dan pengulangan menggunakan `if`, `else`, `switch`, `while`, `do-while`, dan `for`.